Build Phase 2 — Proctor Guide

Your Role

During Phase 2, participants should already have a working app. Your job shifts from app setup to **helping them establish Playwright foundations** and keeping them from getting lost in tooling or flaky tests.

Timeline Check-ins

Time Into Phase	Where They Should Be	Red Flag
----------	----------	----------
10 min	Fixture sanity check done, and have a Phase 2 plan/prompts saved	Still fixing major Phase 1 bugs or no usa
30 min	Installing or configuring Playwright	Still debating whether to test at all
50 min	Writing the first happy-path spec	Still stuck in setup or no spec file yet
1 hour	Running and debugging happy-path test	Test has never been run
1.25 hours	Negative-path test started	Still only discussing happy path with no
1.5 hours	Happy-path test stable and negative-path started (or passing)	No passing Playwright test, or no plan fo

Common Issues

Still Fixing Phase 1

If someone's Phase 1 isn't working yet:

- Help them get the minimum: form → API → response displayed
- Suggest simplifying: drop the frontend framework and use plain HTML + fetch
- Don't let them restart from scratch — fix what they have

Missing or Bad Test Data

Some teams stall because they are inventing test data while debugging tests.

Nudge: "Create a tiny fixture set first: one valid case and two invalid cases. Then write tests against those exact inputs."

Ask them to verify:

- Field names in fixtures match frontend/API names exactly
- Expected outcomes are explicit per fixture (success vs specific error)

Playwright Setup Friction

Watch for people getting stuck on setup details:

- Browser install issues
- Not knowing which command runs the tests
- App and test runner not pointed at the same local URL
- Confusion about where specs should live

Nudge: "Don't optimize the config. Get Playwright running, generate one spec, and prove the happy path works."

Brittle Selectors

The biggest time sink in this phase is fighting selectors that break immediately.

Nudge: "Use `getByRole`, `getByLabel`, and visible text first. Avoid deep CSS selectors unless there's no better option."

Copilot Generating Inconsistent Tests

If Copilot is generating code that doesn't match their existing patterns:

- Use `#file` references to give Copilot context on existing code
- Remind them to review and adjust generated code, not just accept everything
- Ask them to paste the exact failure into chat instead of asking broad questions like "fix my test"

When to Intervene

Do intervene if:

- Someone is stuck on a Phase 1 issue — help them fix it fast so they can move to Phase 2
- Someone has spent 15+ minutes without getting Playwright installed or a spec created
- Someone's selectors are obviously brittle and they keep retrying the same pattern
- Someone's app is broken and they don't realize it
- Someone has a passing happy path but no movement toward a negative path by the 1-hour mark

Don't intervene if:

- They're learning through one or two test failures
- They're trying different assertion styles
- They're moving a little slowly but have a spec running

Nudges

If someone doesn't know what to automate first:

> *"Start with the most demoable success path: valid input goes in, a clear success result comes out."*

If someone is blocked before writing specs:

> *"Run a 5-minute fixture sanity check first, then automate the happy path with the known-good input."*

If someone is fighting selectors:

> *"Can Playwright find this by label, button text, or role instead of a long CSS selector?"*

If someone is moving fast:

> *"Great progress — start your negative-path test now so Phase 3 can be about deeper coverage and debugging."*

If someone has one passing test but extra time:

"Use the remaining time to harden: tighten assertions, clean selectors, and create a clear Phase 3 deeper-scenario ticket."

Phase 2 Success Criteria

At the end of Phase 2, each participant should have:

1. Playwright installed and runnable
2. One happy-path Playwright test passing
3. One negative-path test started (preferably passing)
4. A basic understanding of how to debug selector, timing, or assertion failures

It's OK if: The negative-path test is not fully complete yet, as long as the happy path is stable and the next scenario is clearly defined.

It's not OK if: They finish the phase without a passing Playwright test.

Facilitator One-Page: Variation Matrix (All Phases)

Use this during check-ins to recommend 2-3 experiments per phase without slowing teams down.

Phase	Pick 2-3 Variations	Fast Proctor Cu
---	---	---
Phase 1: working flow first	Test-First Variation, Selector Quality Challenge, Prompt Detail A/B Test	"Get one stable pa
Phase 2: correctness and confidence	Failure Injection Drill, Explain-Back Check, Role-Based Prompting	"Now stress the ap
Phase 3: polish and demo strength	Model Comparison, Refactor Pass, Demo Readiness Variant	"Choose the clean

30-Second Recommendation Script

If a participant asks what to do next, use this:

1. "Pick one reliability variation and one communication variation."
2. "Reliability means selector quality or failure injection."
3. "Communication means explain-back or demo-readiness script."
4. "If you still have time, do model comparison on one prompt."

Decision Rules by Situation

- If they are unstable or flaky: prioritize **Selector Quality Challenge** and **Failure Injection Drill**.
- If they are slow due to rework: prioritize **Prompt Detail A/B Test**.
- If they are close to demo time: prioritize **Refactor Pass** and **Demo Readiness Variant**.
- If they finish early: run **Model Comparison** on one completed checklist prompt and keep the better result.